COFSD: Counter-Only False Sharing Detection

Reducing the Hardware Cost of On-the-fly False-Sharing Detection in Cache-Coherent Multicores

Anil Muddamsetti · Nathan K. Nakamoto · Ronald A. Rodriguez

Group 15 - CSE 220: Computer Architecture · UC Santa Cruz, Baskin School of Engineering

Final Project Report · June 2026

Reference paper: *"Leveraging Cache Coherence to Detect and Repair False Sharing On-the-fly" (MICRO 2024)*

Abstract

False sharing is a silent performance bug: independent variables that land on the same cache line force the coherence protocol to invalidate and re-fetch the line on every cross-core write. FSDetect (MICRO 2024) detects this on-the-fly using fetch/invalidation counters plus byte-granular PAM and SAM tables, but those tables cost roughly 152.5 KB of metadata on an 8-core system and add a MD_REP metadata message on every intervention. **COFSD** keeps FSDetect's FC/IC counters but replaces the byte-level tables with two small per-directory-entry bitmasks (AccessorSet and WriterSet), reducing metadata to 32 bits per entry (~4 KB, a 38x reduction) with no per-core tables and no metadata messages. We validate COFSD in a Python MESI coherence model driven by access patterns taken from the Phoenix linear_regression and string_match benchmarks. COFSD correctly flags both false-sharing benchmarks and neither control, at the cost of one structurally-explained false positive on pure write-write true sharing.

1. Project Plan

Technique: COFSD (Counter-Only False Sharing Detection) is a lightweight redesign of the detection half of the FSDetect/FSLite proposal from the original paper. FSDetect identifies a falsely shared cache line when its fetch counter (FC) and combined invalidation/intervention counter (IC) both cross a privatization threshold τP while no byte-level true sharing is present. To rule out true sharing, FSDetect maintains per-core **PAM** tables (one read and one write bit per byte of every L1 line) and a shared **SAM** table at the directory (last writer and reader set per byte). COFSD keeps FC and IC but discards the per-byte tables, substituting two per-directory-entry bitmasks: **WriterSet**

(one bit per core that has written the line) and **AccessorSet** (one bit per core that has touched the line). A line is flagged when FC and IC cross τP, IC is at least half of FC, and either two or more cores have written the line (write-write false sharing) or exactly one core has written it while three or more cores have accessed it (write-read false sharing).

Implementation approach: We evaluate COFSD in a self-contained Python MESI coherence model rather than a full cycle-accurate simulator. The model carries exactly the protocol state needed to drive the detector a MESI state, owner, and sharer set per line and we feed it the access patterns extracted from the Phoenix benchmark sources. This isolates the detection logic, which is the contribution under test, and keeps the experiment fully reproducible from a single script.

Input the technique requires:

- An access trace of (core id, line address, read/write) tuples. Each tuple is the per-core L1 demand load or store that the detector consumes. From these the model derives the coherence events (GetS / GetX / Upgrade, invalidations, interventions) that drive FC and IC.
- The traces are generated from the Phoenix sources: linear_regression's emit_intermediate() partial-sum writes (write-write pattern) and string_match's str_data_t splitter-write / map-read pattern (write-read), plus a true-sharing lock and a private no-sharing control.

Output the technique generates:

- A per-line metadata record {FC, IC, HC, AccessorSet, WriterSet} maintained as the trace is replayed.
- A detection log: flagged line address, FC/IC at the trigger, classification (write-write or write-read), and the participating cores.
- Aggregate counters per scenario: total accesses, invalidations, invalidation rate, and detection count (printed at the end of the run).

Source files: The implementation is a single module, false_sharing_detector.py, containing the COFSD Simulator class (MESI model plus detector), the four scenario drivers, the τP sensitivity sweep, and the FSDetect-vs-COFSD hardware-cost estimate.

Data structures: All COFSD state is held in dictionaries keyed by cache-line address: FC and IC counters, a saturating hysteresis counter HC (to suppress repeated flagging), and the AccessorSet and WriterSet as Python sets of core ids: the software stand-ins

for the C-bit bitmasks a hardware entry that would pack into 32 bits. Alongside these, the model keeps the MESI state, owner, and sharer set per line. No per-core PAM/SAM tables and no metadata-carrying messages are introduced, which is precisely the cost COFSD aims to avoid.

2. Project Check-In

Source-code implementation (Python MESI model, benchmark drivers, controls, τP sweep, and hardware-cost script):

Repository: https://github.com/amuddamse/cofsd-false-sharing.git

Key files in the repository:

- false_sharing_detector.py: the COFSDSimulator MESI model, the two Phoenix benchmark drivers (linear_regression, string_match), the two controls (true-sharing lock, private no-sharing), the τP sensitivity sweep, and the FSDetect-vs-COFSD hardware-cost estimate.
- results.txt: captured console output of the full run (seed 42) reproducing every number cited in this report.
- slide_data.txt: the presentation result tables generated from the same run.

3. Final Report

3.1 Background and the COFSD technique

False sharing arises when two or more threads write disjoint bytes of the same cache line. Because an invalidation-based MESI protocol enforces SWMR (single-writer/multiple-reader) at line granularity, each such write invalidates the other cores copies, and the line ping-pongs between caches even though no data is actually shared. The bug produces no incorrect result but only lost performance and wasted interconnect traffic and the reference paper reports an average 1.34x speedup (up to 3.06x peak speedup w.r.t reference count benchmark) simply from removing it manually.

FSDetect detects this online by watching coherence traffic. Its insight is that a falsely shared line generates a recognizable signature: a high fetch count (FC) together with a high invalidation/intervention count (IC), in the absence of byte-level true sharing. To distinguish true from false sharing it tracks per-byte read/write metadata in per-core PAM tables and a directory-side SAM table. That precision is what costs hardware: on the 8-core configuration we model, PAM tables total ~64.5 KB and SAM tables ~88 KB, for roughly 152.5 KB of metadata (~0.9 % of LLC capacity), plus a REP_MD metadata message on each qualifying intervention.

COFSD asks whether the FC/IC counters alone, augmented with only a coarse, per-core (not per-byte) view of who has touched and who has written the line, are enough to catch most harmful false sharing. It replaces the byte-level tables with two bitmasks per directory entry, giving the detection condition below.

Signal	COFSD detection rule
Counters	FC ≥ τP AND IC ≥ τP AND IC ≥ 50% of FC (sustained ownership contention)
Write-write FS	WriterSet > 1 - two or more cores have written the line
Write-read FS	WriterSet = 1 AND AccessorSet > 2 - one writer, several readers on the same line
Per-entry cost	FC(7b) + IC(7b) + HC(2b) + AccessorSet(8b) + WriterSet(8b) = 32 bits; No PAM/SAM, No REP_MD

3.2 Implementation

We implemented COFSD as a Python MESI coherence model (COFSDSimulator). Each cache line carries a MESI state, an owner, and a sharer set. The model raises exactly the coherence messages real hardware would on each load/store and updates FC, IC, AccessorSet, and WriterSet from those messages. The single most important correctness fix during development was the treatment of '*silent writes*': when a core already holds a line in M state, its subsequent writes generate no GetX and must not increment FC or IC. An earlier version counted every write as a coherence event, which inflated the counters and produced spurious detections, restricting increments to genuine GetS / GetX / invalidation / intervention messages is what makes the detector track real contention.

Benchmark access patterns are taken from the Phoenix sources. In *linear_regression*, map threads write their partial sums (SX, SXX, SY, SYY, SXY) into adjacent entries of a shared intermediate array in emit_intermediate(), adjacent threads entries share a 64-byte line, producing write-write false sharing. In *string_match*, the splitter writes the bytes_comp field of a str_data_t struct while map threads read neighbouring fields (lengths, file pointers) on the same line, producing write-read false sharing. Two controls bound the detector: a true-sharing lock (all cores write the same byte) and a no-sharing case (each core owns a 4 KB-separated line).

3.3 Evaluation methodology

All experiments use an 8-core configuration, 64-byte lines, and $\tau P = 16$, with a fixed random seed for reproducibility. Each scenario runs 200 iterations of its access pattern with thread order shuffled per iteration. For every scenario we record total accesses, total invalidations, invalidation rate, the number of COFSD detections, and the FC/IC values at the first trigger. Correctness is judged exactly as the assignment specifies: the two false-sharing benchmarks must be flagged, and the two controls must not. We additionally sweep τP over {4, 8, 16, 32, 64} to test threshold sensitivity, and compute an analytic hardware-cost comparison against FSDetect. Because COFSD is a detection technique (the FSLite repair / PRV state is out of scope here), the evaluation targets detection coverage, false-positive behaviour, and hardware cost rather than wall-clock speedup.

3.4 Results

Detection accuracy: COFSD flags both false-sharing benchmarks and the private control correctly, the lone miss is a false positive on the true-sharing lock, discussed in 3.5.

Scenario	FS type	Accesses	Inv. rate	Detected	Correct?
linear_regression	write-write	8000	24.5%	5	**Yes ✓**
string_match	write-read	3000	53.1%	1	**Yes ✓**
true_sharing (lock)	true sharing	3200	50.0%	1	**False pos.**
no_sharing (private)	none	1600	0.0%	0	**Yes ✓**

At the first trigger both benchmarks show FC = 17, IC = 16 just past $\tau P = 16$, confirming the threshold is well-calibrated rather than firing on noise. linear_regression is flagged five times (multiple distinct contended lines, first writers {1,2,3}), string_match is flagged

once, with its 53.1% invalidation rate reflecting the splitter and map threads fighting over one line.

Hardware cost: Replacing the byte-level tables with 32 bits per entry is the central result.

Component (8-core)	FSDetect	COFSD	Savings
PAM tables (per-core)	64.5 KB	-	-64.5 KB
SAM tables (per LLC slice)	88.0 KB	-	-88.0 KB
Directory bits / entry	20 bits	32 bits	+12 bits
REP_MD metadata messages	Required	None	Eliminated
Total metadata	**152.5 KB**	**~4.0 KB**	**38x less**
% of LLC	0.9%	0.02%	**~38x**

τP sensitivity: Detection counts are flat across the swept thresholds: linear_regression = 5 and string_match = 1 at every τP, and the single true-sharing false positive persists at every τP as well. The false positive is therefore structural, inherent to counter-only detection, not an artifact of threshold choice, raising τP only delays detection without removing it.

3.5 Analysis, limitations, and performance discussion

The one false positive is the honest cost of dropping byte-level metadata. When all eight cores write the *same* variable (a lock), WriterSet fills to [0–7], which is indistinguishable from eight cores writing disjoint bytes, the exact case FSDetect's PAM/SAM tables exist to disambiguate.

COFSD cannot confirm byte-level disjointness, so it occasionally misclassifies pure write-write true sharing as false sharing. The runtime impact is bounded: a false flag would cause at most one extra privatization attempt, which a real protocol terminates on the first true-sharing conflict, and the saturating hysteresis counter (HC) suppresses repeated re-flagging of the same line. Pure write-write true sharing is also comparatively rare in practice.

On performance: COFSD is a detection mechanism, so it is not expected to produce a speedup on its own. The runtime benefit of false-sharing repair belongs to FSLite's

privatization, which we did not implement. Measured against its actual goal, COFSD succeeds: it preserves FSDetect's detection coverage on both the write-write and write-read benchmarks while cutting detection metadata by roughly 38x and removing the per-intervention metadata messages entirely. The appropriate verdict, in the assignment's own terms, is that the technique meets its design objective (cheaper detection at equal coverage) and that its single failure mode is understood and explainable rather than incidental.

Is COFSD the better choice: Especially for area-constrained designs running workloads dominated by write-write or write-read false sharing, where a rare, self-correcting false positive is acceptable, COFSD is a better choice. But, when you need a byte-level precision, mixed true/false sharing on the same lines, or settings that require zero false positives FSDetect is needed and it is a better choice.

3.6 Conclusion and future work

COFSD shows that the bulk of harmful false sharing can be caught with simple per-directory-entry counters and bitmasks, without the costly hardware overhead of per-byte PAM/SAM tables that dominate FSDetect's area. In our MESI model it detects both Phoenix false-sharing benchmarks and neither benign control, using about 4 KB of metadata in place of 152.5 KB. The future work that can be carried is to (i) port the detector into a cycle-accurate simulator such as Scarab to confirm the results on full simulated runs with real timing, (ii) pair COFSD detection with FSLite-style privatization (the PRV state) to turn detection into repair and measure speedup directly, and (iii) reduce the false-positive class with a minimal per-byte epoch bit, trading a small, bounded area increase for cleaner true/false discrimination.

References

[1] V. Patel, S. Biswas, and M. Chaudhuri, “Leveraging Cache Coherence to Detect and Repair False Sharing On-the-fly,” in Proc. 57th IEEE/ACM Int. Symp. Microarchitecture (MICRO), 2024, pp. 823–839.

[2] V. Nagarajan, D. J. Sorin, M. D. Hill, and D. A. Wood, A Primer on Memory Consistency and Cache Coherence, 2nd ed. Morgan & Claypool, 2020.

[3] M. S. Papamarcos and J. H. Patel, “A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories,” in Proc. Int. Symp. Computer Architecture (ISCA), 1984, pp. 348–354.

[4] C. Ranger, R. Raghuraman, A. Penmetsa, G. Bradski, and C. Kozyrakis, “Evaluating MapReduce for Multi-Core and Multiprocessor Systems,” in Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA), 2007, pp. 13–24. Phoenix shared-memory MapReduce benchmark suite; source obtained via git clone https://github.com/kozyraki/phoenix.git (the linear_regression and string_match access patterns used in this work are derived from this suite).

[5] G. Venkataramani, C. J. Hughes, S. Kumar, and M. Prvulovic, “DeFT: Design Space Exploration for On-the-Fly Detection of Coherence Misses,” ACM Trans. Architecture and Code Optimization (TACO), vol. 8, no. 2, 2011.

[6] M. Chabbi, S. Wen, and X. Liu, “Featherlight On-the-Fly False-Sharing Detection,” in Proc. ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP), 2018, pp. 152–167.

[7] T. Liu and E. D. Berger, “SHERIFF: Precise Detection and Automatic Mitigation of False Sharing,” in Proc. ACM OOPSLA, 2011, pp. 3–18.

[8] T. A. Khan, Y. Zhao, G. Pokam, B. Mozafari, and B. Kasikci, “Huron: Hybrid False Sharing Detection and Repair,” in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 2019, pp. 453–468.

[9] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, et al., “The gem5 Simulator,” ACM SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1–7, 2011.

[10] C. Bienia, S. Kumar, J. P. Singh, and K. Li, “The PARSEC Benchmark Suite: Characterization and Architectural Implications,” in Proc. Int. Conf. Parallel Architectures and Compilation Techniques (PACT), 2008, pp. 72–81.

[11] V. Gramoli, “More Than You Ever Wanted to Know about Synchronization: Synchrobench, Measuring the Impact of the Synchronization on Concurrent Algorithms,” in Proc. PPoPP, 2015, pp. 1–10.

[12] HPS Research Group, “Scarab Microarchitectural Simulator.” [Available Online]: https://github.com/Litz-Lab/scarab

[13] HPS Research Group, “Scarab Microarchitectural Docker File.” [Available Online]: https://github.com/Litz-Lab/Scarab-infra/tree/cse220